

Proyecto de Software

Directivas Vue

Directivas Vue

- Son atributos específicos de Vue que comienzan con **v-**.
 - **v-bind**, **v-model**.
 - Condicionales: **v-if**, **v-else**, **v-else-if**, **v-show**.
 - Bucles: **v-for**.
 - Eventos: **v-on**.
 - Otras: **v-text**, **v-html**, **v-once**. Ver [todas las directivas](#).
- Las más usadas tienen **abreviatura**: `v-bind:title` → `:title` y `v-on:click` → `@click`.

Directiva **v-bind**:

- La interpolación `{{}}` no funciona para atributos, se utiliza **v-bind** para relacionar atributos con datos de Vue. Veamos ejemplo **v-bind**.

```
<div id="app">
  <span v-bind:title="message">Hover your mouse over me !!</span>
</div>
<script>
  const App = {
    data() {
      return { message: 'Fecha ' + new Date().toLocaleString() }
    }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

- Abreviado: `<span :title="message">`.

Directiva condicional v-if:

- Veamos v-if.

```
<div id="app">
  <p>
    <span v-if="seen">Now you see me</span>
    <span v-else>Oh no 😱</span>
  </p>
</div>
<script>
  const App = {
    data() { return { seen: true } }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

- **v-show** también muestra u oculta, pero el elemento queda en el DOM (sólo cambia `display`). Conviene si se alterna seguido.

Bucles v-for:

- Veamos **v-for**. Modifiquemos **vue_app.products**, agregando: **vue_app.products.push("Manteca")** y eliminando: **vue_app.products.pop()**.

```
<div id="app">
  <ul><li v-for="product in products">{{ product }}</li></ul>
</div>
<script>
  const App = {
    data() { return { products: ['Harina', 'Arroz', 'Yerba'] } }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

- Se recomienda agregar **:key** con un valor único, para que Vue identifique cada elemento: **<li v-for="p in productos" :key="p.id">**.

Métodos y eventos v-on:

- La directiva **v-on** nos permite actuar cuando se produzca algún **evento DOM**.
- Dentro de la sección **methods** ponemos el método que se va a disparar cuando el evento se produzca. Veamos **v-on**.

```
<div id="app">
  <p>{{ message }}</p>
  <button v-on:click="reverseMessage">Reverse Message</button>
</div>
<script>
  const App = {
    data() { return { message: 'Bienvenidos a Proyecto de Desarrollo!!!' } },
    methods: {
      reverseMessage() {
        this.message = this.message.split('').reverse().join('')
      }
    }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

Eventos v-on:

- Incluso es posible "colgarse" de múltiples eventos:

```
<div v-on="{  
  click    : onClick,  
  keyup    : onKeyUp,  
  keydown  : onKeyDown  
}">  
</div>
```

- Notar que se modifica el estado de nuestra aplicación sin tocar el DOM, todo eso lo hace Vue.
- El código queda simplificado y enfocado en la lógica de lo que hay que resolver.

Directiva **v-model**:

- Hace la relación bidireccional entre un input y los datos de la aplicación Vue.
- Veamos **v-model** y **v-models**.

```
<div id="two-way-binding">
  <p>{{ message }}</p>
  <input v-model="message">
</div>
<script>
  const App = {
    data() { return { message: 'Bienvenidos a Proyecto!!!' } }
  }
  vue_app = Vue.createApp(App).mount('#two-way-binding')
</script>
```

Propiedades computadas:

- Veamos [propiedades-computadas](#).
- Nos evita poner demasiada lógica en la visualización. Se recalculan sólo cuando cambian los datos de los que dependen.

```
<div id="app">
  <ul><li v-for="product in products">{{ product }}</li></ul>
  Cantidad de elementos: {{ countProducts }}
</div>
<script>
  const App = {
    data() { return { products: ['Harina', 'Arroz', 'Yerba'] } },
    computed: {
      countProducts() { return this.products.length }
    }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

Watchers:

- Veamos [watcher](#). Para reaccionar cuando un dato cambia.
- Para calcular un valor conviene **computed**. Un **watcher** se usa para efectos secundarios: llamar a una API, guardar en `localStorage`, etc.

```
<div id="app">
  <ul><li v-for="product in products">{{ product }}</li></ul>
  Cantidad de elementos: {{ countProducts }}
</div>
<script>
  const App = {
    data() { return { products: ['Harina', 'Arroz', 'Yerba'], countProducts: 3 } },
    watch: {
      products: {
        handler() { this.countProducts = this.products.length },
        deep: true
      }
    }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

Consumiendo una API con Vuejs

Ejemplo básico consultando una API con fetch

- Veamos [lista-api](#). En este caso utilizamos el hook **created** dentro del **ciclo de vida** de la instancia Vue (lo vemos al final).

```
<div id="app">
  Datos de: https://jsonplaceholder.typicode.com/
  <ul><li v-for="post in posts">{{ post.id }} - <b>{{ post.title }}:</b> {{ post.body }}</li></ul>
</div>
<script>
  const App = {
    data() { return { posts: [] } },
    created() {
      fetch('https://jsonplaceholder.typicode.com/posts')
        .then(response => response.json())
        .then(json => { this.posts = json })
    }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

Utilizando el cliente HTTP **axios**

- Realiza las peticiones HTTP desde el navegador (con **XMLHttpRequest** o **fetch**) y desde Node.js (servidor).
- Soporta API de **Promesas de JS**.
- Intercepta y transforma los datos de requerimientos y respuestas.
- Transforma automáticamente a JSON.
- Soporte del lado del cliente para protección contra **CSRF**.
- Documentación: axios-http.com.

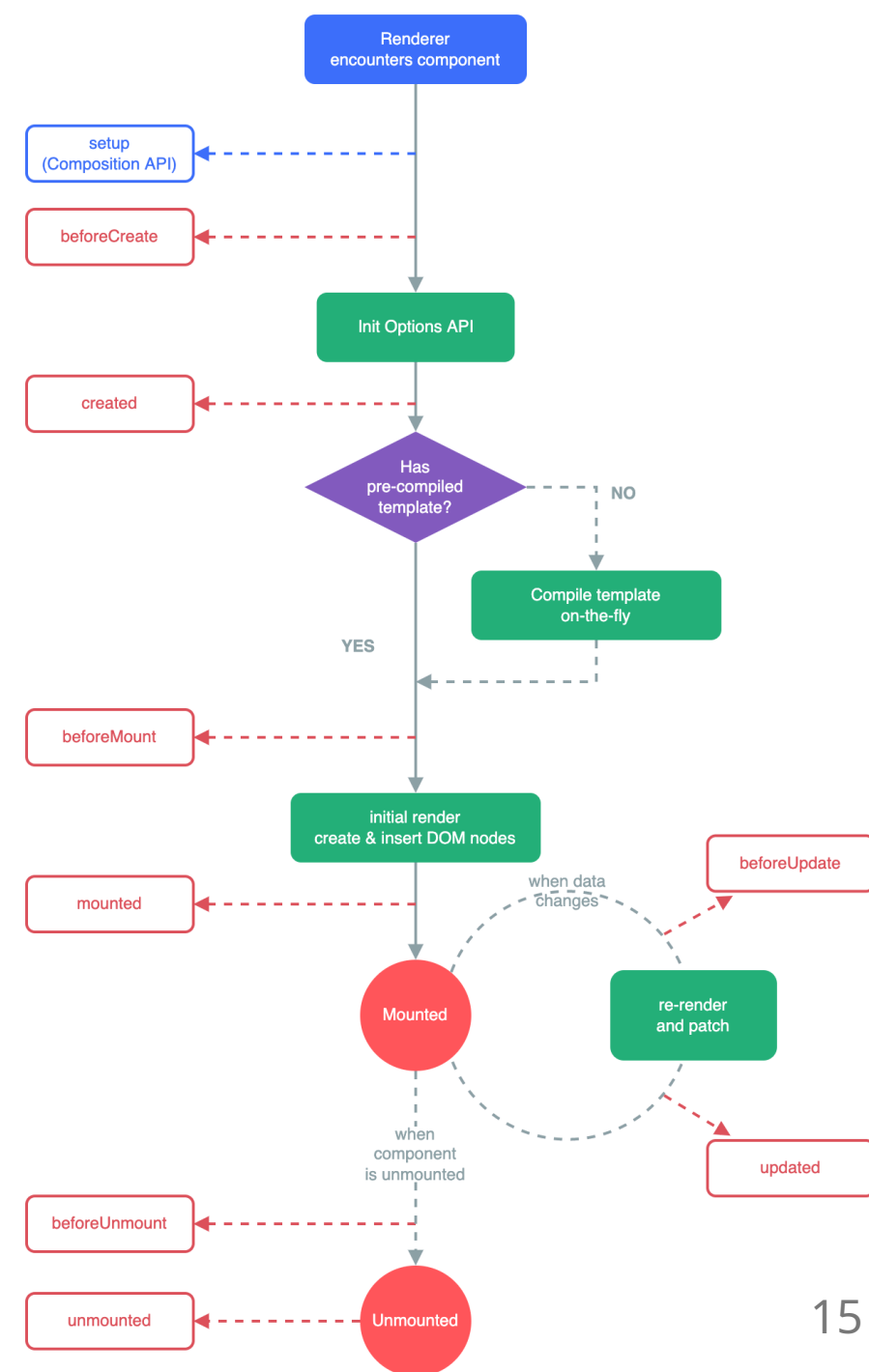
Ejemplo básico consultando una API con axios

- Veamos [lista-api-axios](#). Ahora, abramos directamente el archivo local en el navegador.
- Se incluye con `<script src="https://unpkg.com/axios/dist/axios.min.js"></script>`.

```
<div id="app">
  <h1>Provincias Argentinas:</h1>
  <ul><li v-for="p in provincias">{{ p.id }} - {{ p.nombre }}</li></ul>
  <ul v-if="errors.length"><li v-for="error of errors">{{ error.message }}</li></ul>
</div>
<script>
  const App = {
    data() { return { provincias: [], errors: [] } },
    created() {
      axios.get('https://apis.datos.gob.ar/georef/api/provincias')
        .then(response => { this.provincias = response.data.provincias }) // ya es JSON
        .catch(e => { this.errors.push(e) })
    }
  }
  vue_app = Vue.createApp(App).mount('#app')
</script>
```

Ciclo de vida de una instancia Vue

- Vue ejecuta **hooks** en momentos clave de la vida de una instancia o componente:
 - **created**: datos reactivos, todavía sin DOM. Buen lugar para pedir datos a una API.
 - **mounted**: ya está en el DOM. Para acceder a elementos o integrar librerías que lo manipulan.
 - **beforeUpdate** / **updated**: antes y después de re-renderizar por un cambio de datos (*re-render and patch*).
 - **unmounted**: se quitó del DOM. Para limpiar timers o *listeners*.
- Con Composition API se usan `onMounted()`, `onUnmounted()`, etc.
- Ver [diagrama y guía oficial](#).



Referencias Vue

- [Guía oficial de Vue](#) y [tutorial interactivo](#).
- [Sintaxis de templates](#) y [referencia de directivas](#).
- [Ejemplos oficiales](#) y [Playground online](#).
- [Vue Mastery: Intro to Vue 3](#) (curso en video, en inglés).
- [Vue DevTools](#) para depurar en el navegador.

¿Dudas?

Seguimos con vuejs la próxima semana...